

Assignment 3

[6pt] 2025.12.01

A. Control Theory

(a) Find the natural frequency ω_n and the natural damping ratio ζ_n of the natural system, i.e., $f = 0$. What kind of system is this (oscillatory, overdamped, ...)?

Solution:

Given the system

$$16\ddot{x} + 16\dot{x} + 4x = 0,$$

we have the parameters

$$m = 16, \quad b = 16, \quad k = 4.$$

We can calculate the natural frequency with the formula:

$$\omega_n = \sqrt{\frac{k}{m}}.$$

Substituting the values,

$$\omega_n = \sqrt{\frac{4}{16}} = \sqrt{\frac{1}{4}} = \frac{1}{2}.$$

For the natural damping ratio we have

$$\zeta_n = \frac{b}{2\sqrt{km}}.$$

And therefore

$$\zeta_n = \frac{16}{2\sqrt{4 \cdot 16}} = \frac{16}{2\sqrt{64}} = \frac{16}{2 \cdot 8} = 1.$$

The system is critically damped since $\zeta_n = 1$.

(b) Design a PD controller f that realizes a critically damped system with a closed-loop stiffness of $k_{CLS} = 16$. Assume that the desired position is $x_d = 0$.

Solution:

We write the dynamics of the system as

$$16\ddot{x} + 16\dot{x} + 4x = f.$$

given the desired position $x_d = 0$ we have the PD law

$$f = -k_v\dot{x} - k_px,$$

We can insert the controller into the system equation giving us the closed-loop dynamics

$$16\ddot{x} + (16 + k_v)\dot{x} + (4 + k_p)x = 0.$$

Closed-loop stiffness:

$$4 + k_p = k_{CLS} = 16,$$

$$k_p = 12.$$

Critical damping:

$$16 + k_v = 2\sqrt{m k_{CLS}} = 2\sqrt{16 \cdot 16} = 32.$$

$$k_v = 16.$$

For $k_{CLS} = 16$ we therefore end up with the controller $f = -16\dot{x} - 12x$.

(c) Assume that the friction changes from linear friction ($b = 16\dot{x}$) to Coulomb friction:

$$b = 30 \cdot \text{sign}(\dot{x}).$$

Design a controller that uses a non-linear model-based portion with trajectory following to critically damp the system at all times and maintain a closed-loop stiffness of $k_{CLS} = 16$. In other words, let

$$f = \alpha f' + \beta, \quad f' = \ddot{x}_d - k'_v(\dot{x} - \dot{x}_d) - k'_p(x - x_d).$$

Note that f is an m -mass control, while f' is a unit-mass control. Use the definition of error $e = x - x_d$.

Solution:

We update our system dynamics with the Coulomb friction so that

$$16\ddot{x} + 30 \text{sign}(\dot{x}) + 4x = f.$$

We have to pick α and β in our partitioned control function $f = \alpha f' + \beta$ such that the closed-loop error dynamics take the form of a critically damped unit-mass system.

To recover unit-mass dynamics from the f' input, we choose

$$\alpha = m = 16, \quad \beta = 30 \text{sign}(\dot{x}) + 4x.$$

Using the error variable $e = x - x_d$, the unit-mass controller is defined as

$$f' = \ddot{x}_d - k'_v(\dot{x} - \dot{x}_d) - k'_p(x - x_d).$$

Then we can insert $f = \alpha f' + \beta$ into the system equation which gives us

$$16\ddot{x} + 30 \text{sign}(\dot{x}) + 4x = 16(\ddot{x}_d - k'_v(\dot{x} - \dot{x}_d) - k'_p(x - x_d)) + 30 \text{sign}(\dot{x}) + 4x.$$

The model-based term β cancels the nonlinear friction and spring forces, leading to the closed-loop error dynamics

$$16\ddot{e} + 16k'_v\dot{e} + 16k'_pe = 0.$$

Matching the desired closed-loop stiffness:

$$16k'_p = k_{CLS} = 16,$$

$$k'_p = 1.$$

Critical damping condition for our second-order system:

$$16k'_v = 2\sqrt{m k_{CLS}} = 2\sqrt{16 \cdot 16} = 32,$$

$$k'_v = 2.$$

Final unit-mass controller:

$$f' = \ddot{x}_d - 2(\dot{x} - \dot{x}_d) - 1(x - x_d).$$

We end up with the control-law $f = 16f' + 30 \text{sign}(\dot{x}) + 4x$.

(d) What is the steady-state error $e = x - x_d$ of the system in part (c) (i.e., when $\ddot{e} = \dot{e} = 0$) if it is disturbed by a constant force $f_{\text{dist}} = 8$?

Solution:

From the closed-loop error dynamics obtained in part (c) we have,

$$16\ddot{e} + 16k'_v\dot{e} + 16k'_p e = 0,$$

a constant disturbance force f_{dist} enters the steady-state force balance. At steady state we have $\dot{e} = \ddot{e} = 0$, so only the stiffness term remains, giving

$$k_{CLS} e = f_{\text{dist}}.$$

With the closed-loop stiffness $k_{CLS} = 16$ and disturbance $f_{\text{dist}} = 8$, the steady-state error becomes

$$e = \frac{f_{\text{dist}}}{k_{CLS}} = \frac{8}{16} = \frac{1}{2}.$$

Thus, the system settles with a steady-state position error of $e = \frac{1}{2}$.

B. Visual Servoing

1. Image Features

Object 1: Ellipse

Extraction. An ellipse can be extracted by detecting its contour and applying an ellipse fitting algorithm (e.g. OpenCV's `fitEllipse`). The extraction is moderately difficult and somewhat sensitive to noise because an ellipse requires a stable closed contour.

Object Parameters ($k = 5$).

- u, v : center of the ellipse in the image (pixels)
- a, b : lengths of the major and minor axes
- α : orientation of the major axis (0° – 180°)

Minimal parameterization. An ellipse has five degrees of freedom, so the chosen parameter set is already minimal.

Controllable DoFs ($m = 4$). The ellipse provides sufficient geometric structure to control:

x, y, z translation and z -axis rotation.

Image Jacobian dimensionality.

$$J_I \in \mathbb{R}^{5 \times 4}.$$

Constraint structure. Since $k > m$, the system is **overconstrained**. A pseudoinverse must be used to handle the redundant feature information.

Object 2: Two Parallel Lines

Extraction. Two line segments can be detected robustly using Canny edge detection and a Hough transform. Extraction is relatively simple, provided both lines are visible.

Object Parameters ($k = 4$).

- d : pixel distance between the two lines
- θ : orientation of the lines (0° – 180°)
- u, v : midpoint location between the two lines

Minimal parameterization. Two parallel lines are uniquely defined by four degrees of freedom (position, spacing, orientation).

Controllable DoFs ($m = 4$). The object allows control of:

x, y, z translation and z -axis rotation.

Using two lines removes the directional ambiguity of a single infinite line.

Image Jacobian dimensionality.

$$J_I \in \mathbb{R}^{4 \times 4}.$$

Constraint structure. Since $k = m$, the system is **well-formed**.

Object 3: Cross Marker

Extraction. A cross marker (“+”) is very robust to noise and illumination changes. Extraction only requires detecting two perpendicular segments and their intersection.

Object Parameters ($k = 3$).

- u, v : intersection point (image coordinates)
- s : scale of the cross (e.g. average length of horizontal and vertical arms)

Minimal parameterization. Since the cross shape is fixed, only position and scale are needed, yielding a minimal 3-parameter representation.

Controllable DoFs ($m = 3$). The cross marker cannot provide orientation about the z -axis due to its fourfold symmetry. It therefore supports control of:

x, y, z translation.

Image Jacobian dimensionality.

$$J_I \in \mathbb{R}^{3 \times 3}.$$

Constraint structure. Since $k = m$, the system is again **well-formed**.

Object	k	m	Jacobian Size	System Type
Ellipse	5	4	5×4	Overconstrained
Two Parallel Lines	4	4	4×4	Well-formed
Cross Marker	3	3	3×3	Well-formed

2.Find Circle

(a)

To ensure stable circle detection under noisy or adverse imaging conditions, several pre-processing steps were applied before running the Hough Circle Transform:

- **Gaussian smoothing:** A 9×9 Gaussian filter is applied to the backprojected mask to suppress noise and stabilize the circular region.
- **Canny edge detection:** Edges are extracted from the smoothed mask, providing a clean and well-defined contour for the Hough transform and reducing false detections.
- **Hough Circle Transform:** Executed on the edge image with tuned parameters to ensure reliable circle detection.
- **Selection of the final circle:** When multiple circles are detected, the one whose center lies closest to the image center is chosen, increasing robustness to background noise.
- **Debug visualization:** All detected circles and the selected final circle are drawn onto the image to assist in verification and parameter tuning.

These steps provide a noise-tolerant and reliable circle detection method suitable for use in the visual servoing loop.

3.Image Jacobian computation

(a)

Due to the fact that the pinhole camera relationship is independent of the position - which means that a change in the 3D scene induces the same change in the 2D image scene regardless of the origin of the change - we can just use the relationship and plug in the diameter values.

$$z = f/d' \cdot d$$

Where d' is the diameter on the image plane calculated by the Hough transformation, d is the real absolute diameter and f is the distance between the pinhole and the image plane, the focal length.

(b)

We could just use again the pinhole camera relationship with a circle as a calibration object - where d' is computed by the hough transformation and d_{real} is given. z can be acquired by measuring the distance between the pinhole and the object. Thus, we can easily calculate the focal length with the following formula.

$$f = d' \cdot z/d$$

However, this has several flaws:

1. it is not robust against noise, one solution could be the make several measurements

with several z , d_{calc} and d . Then we can calculate the average over the several measurements.

2. In practice, it could be hard to actually measure the distance between the pinhole and the object because the pinhole is inside the camera surrounded by case material.

(c)

s , p and q looking as follows.

$$s = \begin{pmatrix} u \\ v \\ d' \end{pmatrix}, p_c = \begin{pmatrix} p_x \\ p_y \\ p_z \end{pmatrix}, q = \begin{pmatrix} q_x \\ q_y \\ q_z \end{pmatrix}$$

Where s is the feature vector in feature frame, p_c the camera position in camera frame and q the position of the object - that we want to track - in camera frame. What we want to derive are the two matrices J_1 and J_2 with the following properties:

$$\dot{\mathbf{q}} = \mathbf{J}_2 \cdot \dot{\mathbf{p}}_c, \dot{\mathbf{s}} = \mathbf{J}_1 \cdot \dot{\mathbf{q}}$$

By multiplying this two matrices we get the so-called image jacobian.

$$\mathbf{J}_i = \mathbf{J}_1 \mathbf{J}_2$$

In order to obtain these two matrices, we have to derive the following relationships.

$$\mathbf{s} = f_1(\mathbf{q}), \dot{\mathbf{s}} = f_2(\dot{\mathbf{q}}), \dot{\mathbf{q}} = f_3(\dot{\mathbf{p}})$$

The first relationship can be derived with the pinhole equation. We can get the following.

$$u = f \cdot \frac{q_x}{q_z}, v = f \cdot \frac{q_y}{q_z}, d' = f \cdot \frac{d}{q_z}$$

For the second relationship, we just need to take the derivative in time on both sides by using the chain-rule and the quotient-rule.

$$\dot{u} = f \cdot \frac{\dot{q}_x q_z - \dot{q}_z q_x}{q_z^2}, \dot{v} = f \cdot \frac{\dot{q}_y q_z - \dot{q}_z q_y}{q_z^2}, \dot{d}' = f \cdot \frac{d}{q_z^2}$$

For the last relationship, we simply take the negative velocity of the camera velocity v .

$$\dot{q} = -v$$

For the construction of $\mathbf{J}_2$ we convert the third relation into a matrix. For that we need the negative identity matrix because we just take the negative of the input.

$$\mathbf{J}_2 = -\mathbf{I}$$

For the construction of $\mathbf{J}_1$ we need to convert the second relationship into a matrix form. When we do that, we obtain the following:

$$\mathbf{J}_1 = \begin{pmatrix} f \frac{1}{q_z} & 0 & f(-\frac{q_x}{q_z^2}) \\ 0 & f \frac{1}{q_z} & f(-\frac{q_y}{q_z^2}) \\ 0 & 0 & -f d \cdot \frac{1}{q_z^2} \end{pmatrix}$$

q_z can be calculated as explained in task 3.a. However, it is impossible to directly measure $\mathbf{q}$ in the camera frame, thus, we need to infer these values out of the known feature current coordinates s . That can be done by using the first relation f_1 and rearrange it to q_x and q_y .

$$q_x = \frac{uq_z}{f}, q_y = \frac{vq_z}{f}$$

Now we can plug that into the matrix $\mathbf{J}_1$ and we get the following.

$$\mathbf{J}_1 = \begin{pmatrix} f \cdot \frac{1}{z} & 0 & -\frac{u}{z} \\ 0 & f \cdot \frac{1}{z} & -\frac{v}{z} \\ 0 & 0 & -fd \cdot \frac{1}{z^2} \end{pmatrix}$$

Now let us compute $\mathbf{J}_1\mathbf{J}_2$ to obtain the image jacobian $\mathbf{J}_i$.

$$\mathbf{J}_i = \mathbf{J}_1\mathbf{J}_2 = \begin{pmatrix} -f \cdot \frac{1}{z} & 0 & \frac{u}{z} \\ 0 & -f \cdot \frac{1}{z} & \frac{v}{z} \\ 0 & 0 & f \cdot d \cdot \frac{1}{z^2} \end{pmatrix}$$

4. Velocity vector transformation

We want the end-effector to point to the circle with the z-vector of it's coordinate frame. From the provided image we can see that when the camera frame is directly placed in front of the end-effector the z-vector of the end-effector corresponds to the z-vector of the image frame but the x and y axis are flipped. We can therefore define the transformation of the camera frame to the end-effector frame with the matrix:

$$\mathbf{A} = \begin{pmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

To to get the velocity vector in the base frame we are given the pose of the end-effector in the base-frame. We have 7D vector containing the rotation of the end-effector represented as a quaternion. To transform the velocity into the base frame we need to multiply it by the rotation matrix which we can get from the quaternion like this:

$$\mathbf{R} = \begin{pmatrix} 1 - 2(q_yq_y + q_zq_z) & 2(q_xq_y - q_zq_w) & 2(q_xq_z + q_yq_w) \\ 2(q_xq_y + q_zq_w) & 1 - 2(q_xq_x + q_zq_z) & 2(q_yq_z - q_xq_w) \\ 2(q_xq_z - q_yq_w) & 2(q_yq_z + q_xq_w) & 1 - 2(q_xq_x + q_yq_y) \end{pmatrix}$$

Where the quaternion is defined as the vector $q = [q_w, q_x, q_y, q_z]$.

5. Workspace Limitation

The base frame of our robot is located at $(0, 0, 0)$. We can therefore calculate the distance of the end-effector to the base as $\|x_{EE}\|_2$ where x_{EE} is the position of the end-effector in base coordinates and $\|\cdot\|_2$ is the l2-norm. To limit the workspace of our robot we can compute the distance of the next desired end-effector position to the base frame and check if it is above the maximum 0.85 meters. If this is the case we set the previous desired position as the current desired position thereby ignoring the new invalid command.